

Library Management System

Java + JDBC + MySQL

A simple console-based Library Management System demonstrating Java OOP, JDBC, MySQL database interaction, CRUD operations, and book issue/return management.

Project Overview

• A console application for managing books and library members. It supports adding/searching books and members, issuing and returning books, checking availability, and automatic database/table setup.

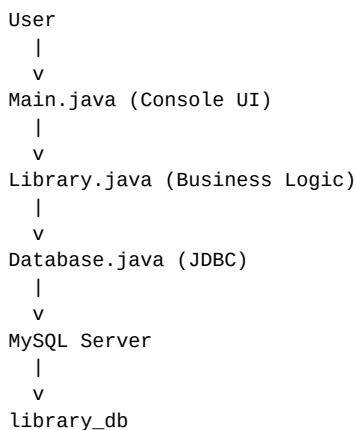
Technologies Used

- Java 21 — application development
- JDBC — Java–MySQL connectivity
- MySQL 8.4 — database
- MySQL Connector/J — JDBC driver
- Command Prompt — compilation/execution
- Git/GitHub — version control and hosting

Features

- 1. Add Book — stores title, author, and quantity.
- 2. View/Search Books — displays books stored in the database.
- 3. Add Member — registers a member with name and phone.
- 4. View/Search Members — displays registered members.
- 5. Issue Book — issues an available copy and records the issue.
- 6. Return Book — records the return and restores availability.
- 7. Check Book Availability — shows available copies.
- 8. Exit — closes the application.

System Architecture



Project Structure

```
LibraryManagementSystem/  
  ■■■ src/  
  ■   ■■■ Main.java  
  ■   ■■■ Library.java  
  ■   ■■■ Book.java  
  ■   ■■■ Member.java  
  ■   ■■■ Database.java  
  ■■■ out/  
  ■■■ mysql-connector-j-26.7.0.jar  
  ■■■ README.md
```

Source Code Structure

Main.java — application entry point, menu, user input, and console interaction.

Book.java — book model containing book-related data.

Member.java — member model containing member-related data.

Library.java — business logic for books, members, issues, returns, and availability.

Database.java — MySQL JDBC connection and database/table initialization.

Database Architecture

```
library_db  
  ■■■ books  
  ■■■ members  
  ■■■ issues
```

books: id, title, author, quantity.

members: id, name, phone.

issues: id, book_id, member_id, issue_date, return_date.

Book Issue Workflow

```
Issue Book  
  |  
Enter Book ID  
  |  
Enter Member ID  
  |  
Check Book + Member  
  |  
Check Availability  
  |  
No copies? -> Reject  
  |  
Create Issue Record  
  |  
Decrease Quantity  
  |  
Issue Successful
```

Book Return Workflow

```
Return Book  
  |
```

```
Find Issue Record
|
Update Return Information
|
Increase Available Quantity
|
Return Successful
```

Availability Logic

- On issue: quantity = quantity - 1
- On return: quantity = quantity + 1
- If quantity reaches 0, another issue is rejected.

JDBC Architecture

```
Java Application
|
v
JDBC API
|
v
MySQL Connector/J
|
v
MySQL Server
|
v
library_db
```

Database Configuration

- Database credentials are supplied at runtime rather than storing the password directly in source code.

Requirements

- Java JDK 21 or compatible JDK.
- MySQL Server running on localhost:3306.
- MySQL Connector/J JAR included in the project.

Project Location

- C:\Mini Projects\LibraryManagementSystem

How to Compile

```
cd /d "C:\Mini Projects\LibraryManagementSystem"

javac -cp ".;mysql-connector-j-26.7.0.jar" -d out src\*.java
```

How to Run

```
java -Ddb.user=root -Ddb.password=YOUR_MYSQL_PASSWORD -cp "out;mysql-connector-j-26.7.0.jar" Main
```

- Never commit a real database password to GitHub.

Example Application Menu

```
=====
      LIBRARY MANAGEMENT SYSTEM
=====
1. Add Book
2. View/Search Books
3. Add Member
4. View/Search Members
5. Issue Book
6. Return Book
7. Check Book Availability
8. Exit
Enter choice:
```

Testing Status

- Java compilation — Passed
- MySQL connection — Passed
- JDBC connection — Passed
- Database setup — Passed
- Add Book — Passed
- View/Search Books — Passed
- Add Member — Passed
- View/Search Members — Passed
- Invalid member handling — Passed
- Issue Book — Passed
- Return Book — Passed
- Availability checking — Passed
- Unavailable-book rejection — Passed
- Exit — Passed

Java Concepts Demonstrated

- Object-Oriented Programming
- Classes and objects
- Constructors
- Encapsulation
- Methods
- Exception handling
- JDBC
- Relational database interaction

Separation of Responsibilities

Main.java	-> User Interface
Library.java	-> Business Logic
Book.java	-> Book Model
Member.java	-> Member Model
Database.java	-> Database Connectivity

Security Note

- This is an educational project. A production system should additionally use secure credential management, authentication, authorization, input validation, environment variables/secrets, transaction management, and audit logging.

Current Limitations

- Console-only interface; no GUI or web interface.
- No authentication system.
- No fine calculation.
- No reservation system.
- No email notifications.
- No multi-branch support.
- No advanced reporting.
- No cloud database.
- No role-based access control.

Future Improvements

- JavaFX/Swing GUI
- Spring Boot web application
- Admin/member authentication
- Fine calculation
- Advanced search
- Reports and analytics
- Reservations
- Notifications
- Role-based access control

Academic Purpose

- This project demonstrates Java programming, Object-Oriented Programming, JDBC, MySQL, database design, CRUD operations, and console-based application development.

Author

Granth Koushik (GK)

License

- This project is intended primarily for educational and academic purposes.